



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Programación Web

Proyecto Final

Alumno:

Tarazón Alvarado Marcos - 22330519
Lizarraga Contreras Raúl Alexander - 23330535
Gracia Mendoza Nicole Alejandra - 22330437

Grupo: S6A

Docente:

Jonathan Moroyoqui

Instituto Tecnológico de Hermosillo.

Hermosillo, Sonora.

26 de Mayo del 2026

Índice.

Introducción.....	3
1. Documentación Técnica.....	4
1.1. Tecnologías utilizadas.....	4
1.2. Arquitectura del SWAB.....	4
2. Distribución del trabajo.....	7
2.1. Tabla de distribución del trabajo en equipo.....	7
3. Retroalimentación del proyecto.....	8
3.1. Marcos.....	8
3.2. Alex.....	8
3.3. Nicole.....	9
4. Retroalimentación del curso.....	9
5. Conclusión.....	12

Introducción.

El proyecto SWAB (Sistema Web de Barbería) es una aplicación web diseñada para apoyar la gestión básica de una barbería de forma más ordenada y práctica. El sistema permite administrar información clave como los servicios disponibles (con precio y duración), el registro de clientes y barberos con datos de contacto, además de ofrecer un flujo sencillo para agendar citas y registrar ventas asociadas a una cita. Con esto se busca evitar el manejo disperso de información (por ejemplo, en notas o archivos sueltos), facilitar la consulta rápida de datos y mantener un control claro de lo que se ofrece y se realiza en la barbería.

La aplicación está construida con una estructura cliente-servidor: un frontend desarrollado con React que presenta la interfaz de usuario y un backend desarrollado con ASP.NET Core Web API que expone endpoints REST para realizar operaciones como crear, consultar, actualizar y eliminar registros. La información se almacena en una base de datos en SQL Server mediante Entity Framework Core, lo que permite persistencia y consistencia en los datos. En conjunto, el proyecto integra las partes esenciales de una aplicación web moderna y funcional, priorizando que sea fácil de ejecutar, probar y demostrar.

1. Documentación Técnica.

1.1. Tecnologías utilizadas.

Para desarrollar el proyecto utilizamos un stack web moderno separando la aplicación en frontend, backend y base de datos. Nos enfocamos en usar herramientas comunes en la industria, fáciles de probar y que nos permitieran entregar un sistema funcional en el tiempo disponible.

Backend (.NET / NuGet)

- .NET: net8.0
- Paquetes NuGet principales:
 - Microsoft.EntityFrameworkCore.SqlServer 8.0.0
 - Microsoft.EntityFrameworkCore.Tools 8.0.27
 - Swashbuckle.AspNetCore 6.6.2 (Swagger)

Frontend (React / npm)

- React: 19.2.6
- Librerías npm principales:
 - react-router-dom 7.15.1 (ruteo/navegación)
- Herramientas:
 - vite 8.0.12 (servidor de desarrollo y build)

Base de datos (SQL Server)

- SQL Server LocalDB (modo local para desarrollo).
Se utiliza para facilitar que cada integrante pueda ejecutar el sistema en su PC sin depender de un servidor externo.

1.2. Arquitectura del SWAB.

Construimos el sistema con una arquitectura cliente-servidor. En términos simples, el frontend muestra la interfaz y envía solicitudes, el backend responde con la lógica y datos, y la base de datos guarda la información de forma persistente.

Estructura del proyecto:

- Backend (ASP.NET Core Web API): ubicado en Barberia.Api/Barberia.Api/
 - Controllers/: controladores con endpoints REST.
 - Models/: entidades del sistema (Servicio, Cliente, Barbero, Cita, Venta).
 - Data/: ApplicationDbContext y configuración de EF Core.
 - Migrations/: migraciones para crear/actualizar el esquema.
 - Program.cs: configuración de servicios, CORS, Swagger y middleware.
- Frontend (React + Vite): ubicado en Barberia.Api/Barberia.Api/vista/
 - src/pages/: pantallas del sistema (Servicios, Clientes, Barberos, Citas, Ventas).

- src/components/: componentes reutilizables (por ejemplo, Navbar).
- src/services/api.js: capa para consumir la API (GET/POST/PUT/DELETE) y evitar hardcodear puertos.

Comunicación Frontend ↔ Backend:

La comunicación se realiza mediante peticiones HTTP/HTTPS hacia endpoints /api/..., enviando y recibiendo datos en JSON. Implementamos CORS para permitir que el frontend (normalmente en localhost:5173) pueda consumir la API aunque el backend se ejecute en otro puerto. Además, utilizamos Swagger para probar endpoints y verificar el correcto funcionamiento del backend durante el desarrollo.

Base de datos:

Para la persistencia de datos definimos entidades que representan los elementos principales de la barbería. La base de datos sigue un modelo sencillo, donde los catálogos alimentan los módulos: primero registramos servicios, clientes y barberos, luego generamos citas, y finalmente registramos ventas ligadas a una cita.

Entidades y atributos principales:

- Servicio: Id, Nombre, Precio, DuracionMin
- Cliente: Id, Nombre, Telefono, Email (Email único)
- Barbero: Id, Nombre, Telefono, Email
- Cita: Id, ClientId, BarberId, ServicioId, FechaHora, Estado
- Venta: Id, CitaId, Monto, FechaHora

Relaciones:

- Una Cita se relaciona con:
 - un Cliente (ClientId)
 - un Barbero (BarberId)
 - un Servicio (ServicioId)
- Una Venta se relaciona con una Cita (CitaId)

API:

Desarrollamos una API REST organizada por recursos, siguiendo una convención estándar de CRUD (GET/POST/PUT/DELETE). Esto nos permite mantener una estructura clara y predecible para el consumo desde el frontend.

Catálogo: Servicios

- GET /api/Servicios → Obtener todos los servicios
- GET /api/Servicios/{id} → Obtener servicio por id
- POST /api/Servicios → Crear servicio
- PUT /api/Servicios/{id} → Actualizar servicio
- DELETE /api/Servicios/{id} → Eliminar servicio

Catálogo: Clientes

- GET /api/Clientes → Obtener todos los clientes

- GET /api/Clientes/{id} → Obtener cliente por id
- POST /api/Clientes → Crear cliente (valida email único)
- PUT /api/Clientes/{id} → Actualizar cliente (valida email único)
- DELETE /api/Clientes/{id} → Eliminar cliente

Catálogo: Barberos

- GET /api/Barberos → Obtener todos los barberos
- GET /api/Barberos/{id} → Obtener barbero por id
- POST /api/Barberos → Crear barbero
- PUT /api/Barberos/{id} → Actualizar barbero
- DELETE /api/Barberos/{id} → Eliminar barbero

Módulo: Citas

- GET /api/Citas → Obtener todas las citas (incluye Cliente, Barbero y Servicio)
- GET /api/Citas/{id} → Obtener cita por id (incluye relaciones)
- POST /api/Citas → Crear cita (no permite fecha en el pasado)
- PUT /api/Citas/{id} → Actualizar cita (no permite fecha en el pasado)
- PATCH /api/Citas/{id}/estado → Cambiar solo el estado de una cita
- DELETE /api/Citas/{id} → Eliminar cita

Módulo: Ventas

- GET /api/Ventas → Obtener todas las ventas (incluye Cita y sus relaciones)
- GET /api/Ventas/{id} → Obtener venta por id
- POST /api/Ventas → Crear venta (ligada a una cita)
- PUT /api/Ventas/{id} → Actualizar venta
- DELETE /api/Ventas/{id} → Eliminar venta

2. Distribución del trabajo.

2.1. Tabla de distribución del trabajo en equipo.

Integrante	Backend (API / BD)	Frontend (React)	Integración / Git
Marcos	Configuramos la base del backend (API en .NET), EF Core, migraciones y conexión a BD. Implementamos y ajustamos validaciones y correcciones de CORS cuando se requirió.	Implementamos la integración inicial de la vista Servicios con la API (crear/listar/eliminar). Ajustamos api.js para evitar errores de Failed to fetch por cambios de puerto.	Realizamos commits de correcciones/validaciones y creamos el PR final del backend hacia main.
Nicole	Apoyo en revisión y pruebas del flujo de endpoints cuando se fue integrando el frontend.	Implementó la estructura del frontend (carpetas, páginas, navegación con Router y componentes). Desarrolló vistas y lógica de Clientes, Barberos y Citas conectadas a la API (formularios, listados y consumo de endpoints).	Subió avances al repositorio mediante commits y pull de cambios para mantenerse sincronizada con develop.
Alex	Apoyó con configuración y ajustes para la ejecución y consumo del backend (CORS y verificación de ejecución desde VS Code/terminal).	Implementó el módulo/vista de Ventas en el frontend (ruta, navegación, registro y listado). Se encargó de aplicar/mejorar el diseño CSS del frontend (estilos globales, navbar, formularios y listas).	Integró cambios en develop (commits) y se coordinó para no generar conflictos durante el cierre del proyecto.

3. Retroalimentación del proyecto.

3.1. Marcos.

Lo más difícil de desarrollar y cómo lo resolví: Lo más complicado fue integrar correctamente el backend con la base de datos y mantener el sistema estable cuando surgían errores de conexión, validaciones y CORS. Lo resolví revisando la configuración del proyecto, ajustando la lógica del API y probando endpoints con Swagger hasta asegurar que las reglas funcionaran.

Herramientas o recursos externos consultados: Documentación oficial de .NET y EF Core, recursos de Swagger y ejemplos de API REST, además de guías rápidas para resolver problemas comunes de CORS y configuración en desarrollo.

Qué cambiaría o mejoraría con más tiempo: Mejoraría el diseño visual del frontend, agregaría validaciones más avanzadas (por ejemplo, evitar choques de horario de citas) y reforzaría el manejo de errores en interfaz para que sea más claro para el usuario.

Qué aprendí que no sabía antes: Aprendí a estructurar un proyecto completo con frontend y backend, a trabajar con migraciones en EF Core y a integrar correctamente una API REST con React.

3.2. Alex.

Lo más difícil de desarrollar y cómo lo resolví: Lo más complicado fue mantener la integración del frontend con el backend cuando el puerto o configuración cambiaba, y dejar un flujo claro en módulos como Ventas. Lo resolví verificando endpoints con Swagger, ajustando el consumo desde React y aplicando cambios para que el sistema fuera más estable en ejecución local.

Herramientas o recursos externos consultados: Documentación de React y Vite, ejemplos de formularios y listados, y guías cortas sobre CORS y consumo de API desde frontend.

Qué cambiaría o mejoraría con más tiempo: Agregaría un diseño más completo y uniforme (mejor UI), y reforzaría validaciones del lado del frontend y backend para casos límite.

Qué aprendí que no sabía antes: Aprendí a integrar módulos del frontend con una API real, a manejar rutas en React y a mejorar la presentación de la interfaz con CSS.

3.3. Nicole.

Lo más difícil de desarrollar y cómo lo resolví: Lo más difícil fue conectar correctamente las vistas del frontend con la API, especialmente cuando había cambios en modelos o validaciones que afectaban los formularios. Lo resolví siguiendo el patrón de consumo de endpoints (GET/POST/DELETE), probando cada vista y ajustando los datos enviados para que coincidieran con el backend.

Herramientas o recursos externos consultados: Documentación de React, uso de React Router y recursos de consumo de APIs con fetch, además de ejemplos prácticos para estructurar páginas y componentes.

Qué cambiaría o mejoraría con más tiempo: Haría el frontend más completo en cuanto a UX (mensajes de éxito/error más claros, mejores formularios), y agregaría edición (PUT) en todas las vistas para tener un CRUD más completo.

Qué aprendí que no sabía antes: Aprendí a estructurar un frontend en React por páginas/componentes y a consumir una API REST de manera ordenada.

4. Retroalimentación del curso.

4.1. Marcos.

Temas del curso más útiles para el proyecto: Lo que más me ayudó fue el desarrollo de la API en Visual Studio, así como el uso de paquetes NuGet para integrar herramientas clave (por ejemplo EF Core y Swagger). Esa parte me dio una base clara para entender cómo estructurar el backend y cómo extender el proyecto con librerías sin reinventar todo desde cero.

Temas que sentí que necesitaban más práctica o explicación: En general, el backend y la integración completa con el frontend fueron los puntos donde sentí que me faltó más práctica. Aunque se cubrieron temas importantes, hubo pocas sesiones para reforzarlos y el ritmo fue rápido, por lo que no alcancé a retener con seguridad todo lo visto en clase.

Qué le agregaría o quitaría al curso: Agregaría más prácticas en clase con un ritmo más pausado y ejercicios más generales, enfocados en dominar cada parte antes de avanzar. También incluiría más tareas de desarrollo para practicar en casa con el entorno web (frontend/back/BD) y así llegar a clase con dudas más específicas. Quitaría la idea de desarrollar un sistema grande completamente en clase, porque consume mucho tiempo y no permite explicar a fondo cada tema o etapa del proyecto.

Calificación del curso (1 al 10) y por qué: 6.5/10. Considero que el curso ofrece una base funcional, pero el ritmo es acelerado. En mi caso aprendí mucho más investigando y practicando por fuera para poder completar el proyecto con seguridad.

Comentarios adicionales para el profesor: El profesor es muy amable y carismático, siempre dispuesto a apoyar a los alumnos. Lo único que mejoraría es el ritmo de desarrollo de los temas, porque en algunos puntos avanzamos demasiado rápido y eso dificulta asimilar bien los conceptos.

4.2. Alex.

Temas del curso más útiles para el proyecto: Para mí, los temas más útiles fueron los relacionados con el frontend, porque fue una parte en la que trabajé bastante durante el proyecto. Me ayudó a comprender mejor cómo se construyen las pantallas, cómo se organiza la interfaz y cómo se conecta con la lógica del sistema.

Temas que sentí que necesitaban más práctica o explicación: El tema que definitivamente necesitaba más explicación fue React, ya que pasamos muy poco tiempo en JavaScript y rápidamente avanzamos a React. En mi caso, esa transición se sintió muy rápida y habría sido útil reforzar primero la base para entender mejor React.

Qué le agregaría o quitaría al curso: Le agregaría más tiempo de práctica en JavaScript, porque dominarlo mejor facilita mucho el aprendizaje de React. Considero que aumentar ejercicios y prácticas enfocadas en JavaScript ayudaría a que el avance a React sea más natural y menos pesado.

Calificación del curso (1 al 10) y por qué: 8/10. En general el curso es útil y aporta una buena base, pero creo que con más práctica previa en JavaScript el aprendizaje de React sería más sólido.

Comentarios adicionales para el profesor: No tengo comentarios adicionales.

4.3. Nicole.

Qué temas del curso les fueron más útiles para el proyecto: Los temas más útiles fueron el desarrollo de APIs, consumo de endpoints desde el frontend, manejo de CRUD y control de versiones con Git, ya que fueron fundamentales para desarrollar e integrar correctamente el sistema.

Qué temas sintieron que necesitaban más práctica o explicación: Considero que hubiera sido útil dedicar más tiempo a la configuración de conexiones entre frontend y backend, manejo de errores, CORS y despliegue del proyecto, ya que fueron temas que requirieron práctica adicional para comprenderlos completamente.

Qué le agregarían o quitarían al curso: Agregaría más prácticas guiadas paso a paso orientadas a proyectos reales, así como ejemplos completos de integración full stack. No quitaría temas, pero sí reorganizará algunos para avanzar de manera más progresiva.

Cómo calificarían el curso del 1 al 10 y por qué: 9/10. Fue un curso muy útil y aplicable, permitió desarrollar habilidades prácticas y reforzar conocimientos importantes. La calificación no es 10 únicamente porque algunos temas técnicos complejos requerían más tiempo de explicación y práctica.

Comentarios adicionales para el profesor: Agradezco la disposición para resolver dudas y la orientación durante el desarrollo del proyecto. Las actividades permitieron aplicar conocimientos reales y fortalecer habilidades importantes para el ámbito profesional.

5. Conclusión.

Como equipo, concluimos que el proyecto SWAB nos permitió aplicar de manera práctica lo visto en el curso para construir una aplicación web completa con frontend, backend y base de datos. Durante el desarrollo logramos integrar un sistema funcional que administra catálogos (servicios, clientes y barberos) y módulos (citas y ventas), manteniendo una estructura clara y fácil de probar mediante endpoints REST y una interfaz web.

Consideramos que sí logramos el objetivo principal del proyecto, ya que el sistema cumple con las funcionalidades requeridas y además implementamos validaciones para mejorar la consistencia de los datos (por ejemplo, evitar citas en el pasado, validar correo y teléfono, correo único en clientes y reglas básicas en servicios). También aprendimos que pequeños detalles de configuración (puertos, CORS, migraciones, etc.) pueden afectar bastante el avance, por lo que fue importante probar constantemente y mantener el repositorio organizado.

Finalmente, como reflexión de trabajo en equipo, entendimos que la comunicación y la división de tareas fueron clave para avanzar sin detenernos. Trabajar con Git y coordinar cambios nos ayudó a integrar el proyecto de forma más ordenada, y aunque hubo retos técnicos, el proceso nos dejó experiencia real sobre cómo se desarrolla un sistema web completo y cómo se resuelven problemas típicos de integración.